Data Retention & Deletion Policy

Version: v1.0 ADOPTED **Adopted:** 2026-07-21 by Sami Ben Chaalia (Security Officer) **Date:** 2026-07-21 **Owner:** Security Officer (Sami Ben Chaalia) **Framework mapping:** SOC 2 CC6.5 / P1 (Privacy) · HIPAA §164.316(b)(2)(i) 6-year retention · GDPR Art. 5(1)(e) storage limitation, Art. 17 right to erasure **Companion documents:** Access Control Policy · Encryption Policy · Incident Response Policy · HIPAA SRA v1 · Privacy Policy (railcall-contrib/website-v2/app/legal/privacy).

1. Purpose

State, per data type, how long RailCall keeps it, why, and how to make it go away. Two separate stories: what RailCall the operator keeps in our own infrastructure, and what the shipped station keeps on the customer's box. Different owners, different rules, different rights.

2. The two stories

- **RailCall-operator side:** we control this data — gateway Postgres rows, Stripe subscription state, Render logs, Anthropic API call metadata (Probo dev only). Retention rules in §3 and §4.
- **Station side (on the customer's box):** the customer controls this — vault, receipts, audit chain, workflow inputs/outputs. Retention rules in §5. This is not our data to delete; we document what the station puts on disk and give the customer commands to remove any of it.

Cross-cutting principle: we retain the MINIMUM amount that satisfies (a) legal + regulatory requirements, (b) our own operational needs (billing integrity, incident forensics), and (c) contractual commitments to customers.

3. RailCall-operator data — retention windows

Data	Where	Retention	Justification
`consumers` row (email, api_key_hash, plan, seat_count, stripe_customer_id)	Render Postgres	Life of account + 6 years after account closure	HIPAA §164.316(b)(2)(i) requires 6 years for records related to compliance. Account closure = deletion request OR 12 months of inactivity, whichever first.
`seat_reservations` row (api_key_hash, install_pubkey_hex, first/last_seen_at)	Render Postgres	Rolling 30 days from last checkin (`_SEAT_TTL_DAYS`)	Pruned automatically by every `/v1/seat/checkin` call. No historical retention needed — enforcement is a snapshot property.
`processed_events` row (event_id, processed_at)	Render Postgres	90 days	Long enough for Stripe retries (72 h max) + our own idempotency window; short enough to keep the table bounded.
Render service logs (gateway HTTP + stdout)	Render (their infra)	Render default (~30 days)	Their retention surface. We don't extend it.
Stripe subscription + payment records	Stripe (their infra)	7 years per Stripe defaults + tax law	Stripe's responsibility surface.
Anthropic API call metadata (from Probo)	Anthropic (their infra)	Anthropic default (30 days, zero-retention on request)	Probo is dev-only — customer data does not reach this path.
Incident notes + post-mortems	Sami's box (`~/incidents/`)	6 years	Same clock as `consumers` rows — an incident affecting a customer is part of that customer's compliance record.

4. Deletion — how each of the above is removed

4.1 On account cancellation (subscription cancelled in Stripe)

Automatic: 1. Stripe fires `customer.subscription.deleted` (or `.updated` with a terminal status). 2. `cloud_gateway.py` webhook sets `consumers.status = 'inactive'` and `seat_count = 0` (cb6ab14, tested by `test_stripe_lifecycle.py`). 3. Consumer is refused for any new mint (§164.312 access control). 4. Seat reservations for that account age out under the 30-day TTL naturally.

The row itself is NOT deleted on cancellation — it stays as an `inactive` record for 6 years per §3. This matters: - **Why we keep it:** proof-of-relationship for regulator or audit inquiry (which paying customer we served, when). - **Why the customer isn't harmed:** all the sensitive material (raw api_key, session tokens) was already hashed or expired at earlier stages; a stale `inactive` row cannot be re-activated without a fresh Stripe subscription minting a new key.

4.2 On explicit customer erasure request (GDPR Art. 17 / "delete my account")

Customer emails `privacy@railcall.ai`. Within 30 days:

1. Verify identity via the customer's registered email + a challenge (activate a signed token from their install; only they can produce it).
2. Locate every row bound to their email + stripe_customer_id: `consumers`, `seat_reservations`, `processed_events` referencing their session ids, any incident notes citing them.
3. Delete the `consumers` row and `seat_reservations` rows outright. Redact `processed_events` and incident notes — replace the customer identifier with a stable pseudonym so referential integrity survives, but nothing traces back to a person.
4. Stripe: request customer deletion via Stripe dashboard — they retain payment records per tax law but detach personal data.
5. Send the customer a confirmation email with a summary of what was deleted, what was retained (Stripe payment records), and why.

Note: HIPAA §164.316(b)(2)(i) 6-year retention CAN legally override GDPR Art. 17 when the customer is a covered entity and the data relates to compliance controls we owe them. In that case: retain the minimum, redact the personal identifiers, and document the retention basis in our reply.

4.3 On issuer seed rotation

Not a deletion event, but tangential: - All entitlement rows in `consumers` remain valid until each install re-mints against the new authority. - Old `signing_pubkey.json` files stay archived on the customer's own box; pre-rotation receipts still verify against the archive (Encryption Policy §7.1).

---

5. Station-side data (on the customer's machine)

RailCall does not read, transmit, or delete these. We document what the station writes so the customer can control their own retention.

Data	Path	Default retention	How to remove
Vault	`~/.railcall/keys.local.json` (CLI) and `<station>/.railcall_workspace/keys.local.json`	Forever until user deletes	`rm -P <path>` (macOS overwrite-then-unlink)
Signing seed	OS keychain (macOS/Windows/Linux) — see Encryption Policy §4.1	Forever until user deletes	`security delete-generic-password -s ai.railcall.signing-seed` on macOS; equivalent on other OSes
Receipts	`~/.railcall/receipts/**`	Forever until user deletes	`rm -rf ~/.railcall/receipts`
Audit chain	`<station>/audit_log.jsonl`, `<station>/integration_audit.jsonl`	Forever until user deletes	`rm <path>` — note this ALSO destroys the chain-based tamper evidence for pre-deletion events
Workflow inputs / outputs	Wherever the customer's workflow writes them	Under the customer's own retention policy	Customer's responsibility
Studio session token	Memory only — process lifetime	Killed on `railcall studio` exit	Kill the process
Entitlement	`<station>/.railcall_workspace/entitlement.json`	Until token expiry or user deletes	`rm <path>`; station reverts to free tier on next check

Customer-side uninstall: `rm -rf ~/.railcall/` removes everything except OS-keychain entries. To remove those too: the `security` command above (macOS) or platform equivalent.

---

6. HIPAA §164.316(b)(2)(i) — the 6-year retention story

HIPAA requires policies + procedures + documentation of compliance-related decisions be retained for 6 years from the later of "creation" or "last effective date." What that means for us:

- **This policy** and every other Probo-tracked policy: retained in Git indefinitely (Git IS the retention layer); a specific version is retained for 6 years from the moment a subsequent version supersedes it.
- **Adopted SRA versions** (`compliance/HIPAA_SRA_v1_2026-07-21.md`): 6 years from adoption date, minimum.

- **Audit records** (incident notes, post-mortems, customer breach notifications): 6 years from the incident close date, minimum.
- **Access review records** (see Access Control Policy §7): 6 years from the review date.
- **Consumer records** (as above): 6 years from account closure.

These are minimums under HIPAA. GDPR erasure requests may reduce these for personal data specifically — see §4.2 for the balancing rule.

---

7. Backup + recovery windows

- **Render Postgres:** Render's managed backup surface — see their SLA. We do not maintain a separate backup.
- **VPS `157.230.177.45`:** DO snapshots weekly (DigitalOcean managed). Snapshots retained for 4 weeks.
- **Issuer seed:** offline backup in 1Password + paper copy in Sami's safe. Retention: same as issuer seed's operational life.
- **Source code:** GitHub (patl4588 org) is authoritative. No off-Git backup — the release tarballs on GitHub Releases + railcall.ai mirror serve as retention for the shipped state at each version.

---

8. What we do NOT retain

Stated explicitly so absence cannot be misread:

- **Customer workflow inputs or outputs.** The engine runs on the customer's box; results never traverse our infrastructure. This is architectural, not policy — see SRA §1 "Key architectural fact."
- **Customer PHI or PII of end-users.** Blocked at construction by `phi_guard` before it can even reach a signed receipt. See Encryption Policy §5.
- **Raw API keys.** Stored as `api_key_hash` (SHA-256) after minting; the raw key is only visible on the success page immediately post-purchase and cannot be re-revealed.
- **Session tokens.** Process-lifetime only; discarded on studio exit.

---

9. Review

At each station release + at least annually. Retention windows are re-checked against current regulatory guidance at each annual review; changes require a corresponding update to the Privacy Policy on `railcall.ai/legal/privacy` at the same time.

---

10. Related documents

- **Privacy Policy** (railcall.ai/legal/privacy) — the customer-facing version of this policy's user rights section.
- **Encryption Policy** §4 — how the data enumerated here is protected while retained.
- **Incident Response Policy** §7.2, §7.3 — retention of incident-related data.

- **BAA_DRAFT.md** — retention obligations we take on when acting as a business associate.